

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	KEERTHANA S	Reg. No.:	192471047
Date:	13.08.2026	Duration:	60 minutes

Code of Conduct

I KEERTHANA S (192471047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Keerthana S

Annexure A – Git & Docker Technical Assignment

Problem Statement : Smart AI-Based Urban Street Lighting Management System

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed
- 6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

1. Problem Analysis

1.1 Problem Statement

Traditional street lighting systems operate on fixed ON/OFF schedules regardless of actual need, which wastes electricity and delays fault detection. Manual inspection cannot keep pace with a large, distributed network of streetlights, so malfunctioning units often remain unnoticed for long periods, compromising public safety. The Smart AI-Based Urban Street Lighting Management System addresses this by combining IoT sensors, cloud infrastructure, and AI models to continuously monitor light status, automatically detect faults, dynamically optimize brightness, and predict maintenance needs through a centralized dashboard.

1.2 Project Objectives

- Continuously monitor the operational status of every streetlight in real time.
- Automatically detect faulty or malfunctioning streetlights.
- Optimize lighting intensity using AI based on traffic, pedestrian, and weather data.
- Reduce overall electricity consumption across the network.
- Predict maintenance requirements before failures occur.
- Generate energy usage and performance reports.
- Improve public safety through reliable, adaptive lighting.
- Support the municipality's smart-city initiatives with centralized, data-driven control.

1.3 Functional Requirements

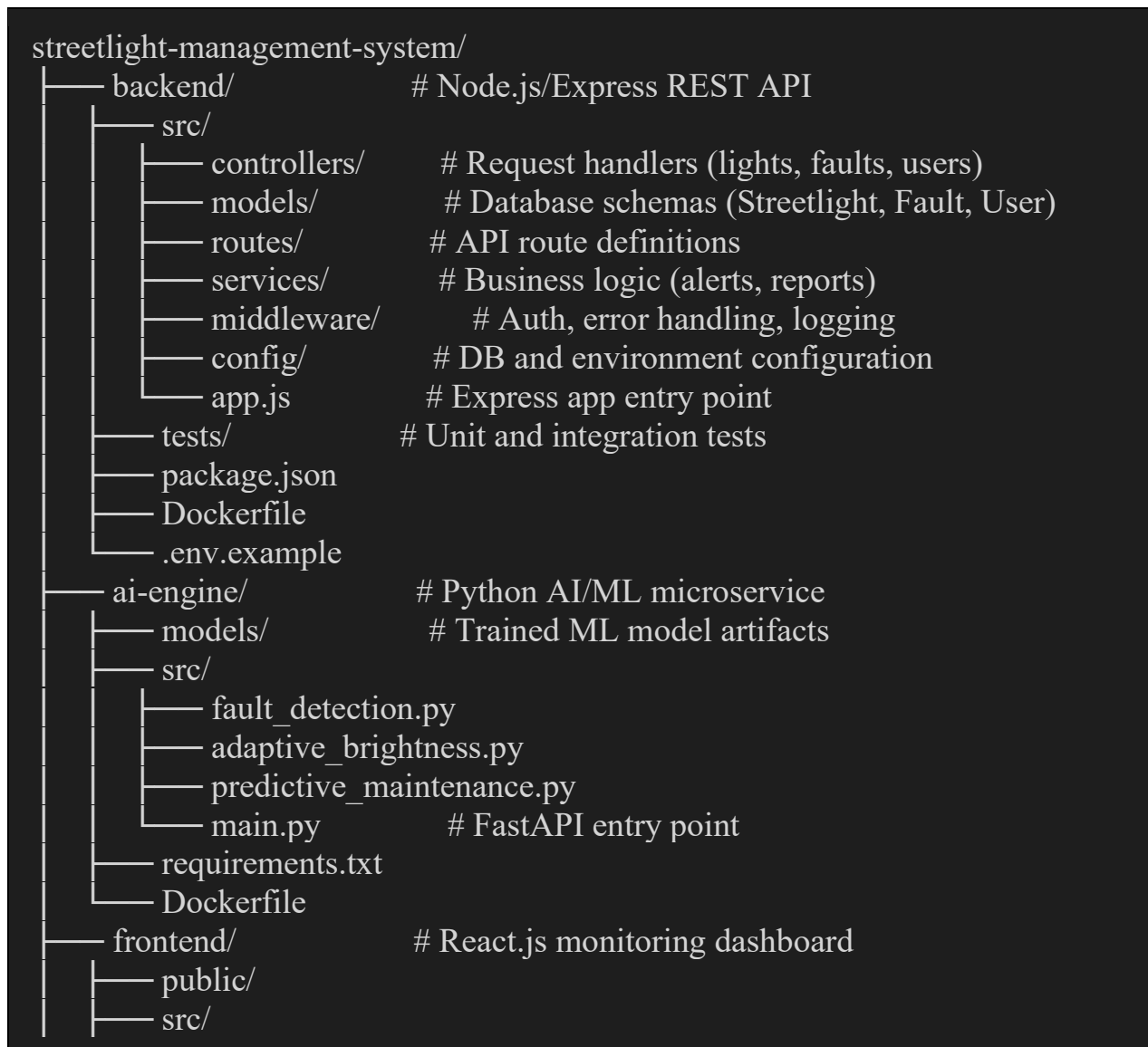
- The system shall ingest periodic status data from IoT-enabled streetlight controllers.
- The system shall detect and classify faults (no output, flickering, communication loss).
- The system shall send real-time alerts to maintenance technicians via SMS/app notification.
- The system shall run an AI/ML engine to adjust brightness based on traffic and weather inputs.
- The system shall provide a web dashboard for monitoring, reporting, and administration.
- The system shall allow citizens to report faults through a mobile application.
- The system shall persist historical data to support predictive maintenance and analytics.

1.4 Expected Outcomes

- Lower energy consumption and reduced maintenance costs.
- Improved streetlight reliability and automated fault detection.
- Enhanced public safety and sustainable city infrastructure.
- Better operational efficiency and increased citizen satisfaction.

2. Project Structure Design

The project follows a microservice-oriented, multi-container architecture. Each major component (backend API, AI engine, frontend dashboard, IoT broker, and database) is isolated into its own directory so it can be developed, tested, versioned, and containerized independently.



```

├── components/      # Map view, fault list, charts
├── pages/           # Dashboard, Reports, Admin
├── services/        # API client wrappers
├── App.js
├── package.json
├── Dockerfile
├── mqtt-broker/
│   └── mosquitto.conf # IoT device communication broker config
├── database/
│   └── init.sql       # Schema and seed data
├── docs/
│   ├── architecture-diagram.png
│   └── workflow-diagram.png
├── screenshots/     # Evidence of Git/Docker operations
├── .gitignore
├── .env.example
├── docker-compose.yml
├── README.md
└── LICENSE

```

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
backend/	Contains the Node.js/Express REST API that handles streetlight data, fault records, and user requests.
ai-engine/	Hosts the Python microservice implementing fault detection, adaptive brightness, and predictive maintenance models, exposed via FastAPI.
frontend/	React single-page application providing the centralized monitoring dashboard used by operators and administrators.
mqtt-broker/	Mosquitto MQTT broker configuration that receives real-time telemetry from IoT streetlight controllers.
database/init.sql	Initializes the PostgreSQL schema (streetlights, faults, users, energy_logs) and seed data on first run.
.gitignore	Excludes build artifacts, node_modules, __pycache__, and secrets from version control.

Folder / File	Purpose
.env.example	Documents required environment variables without exposing actual secrets.
docker-compose.yml	Defines and orchestrates all services (backend, ai-engine, frontend, database, broker) as a single stack.
README.md	Project overview, setup instructions, and usage guide for new contributors.

3. Git Repository Initialization

The Git repository was initialized locally and connected to a remote origin on GitHub to enable collaborative development and backup.

3.1 Initialization Steps

```
# 1. Create the project folder and move into it
mkdir streetlight-management-system && cd streetlight-management-system

# 2. Initialize a local Git repository
git init

# 3. Set the default branch name to 'main'
git branch -M main

# 4. Create essential files
touch README.md .gitignore .env.example

# 5. Stage and make the initial commit
git add .
git commit -m "chore: initial commit - project scaffolding"

# 6. Link the local repository to a remote GitHub repository
git remote add origin https://github.com/<username>/streetlight-management-system.git

# 7. Push the initial commit to the remote
git push -u origin main
```

3.2 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by `git init`; stores the entire object database, refs, and history of the repository.
.gitignore	Lists files/folders (node_modules/, __pycache__/, .env, dist/, *.log) that Git should never track.
.env.example	Template listing required environment variables (DB_URL, MQTT_BROKER_URL, JWT_SECRET) without real values.
README.md	Describes the project, setup instructions, architecture, and how to run it locally or via Docker.

4. Git Branching Strategy

The project adopts the Gitflow branching model, which is well suited to a multi-service system with scheduled releases and a need for stable production code alongside active feature development.

4.1 Branch Types

Branch	Purpose
main	Always reflects the stable, production-ready state of the application. Only release and hotfix branches merge here.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Short-lived branches for individual features, e.g. feature/fault-detection, feature/ai-brightness-control, feature/citizen-app-api.
release/*	Branches cut from develop to stabilize a version (final testing, version bump, docs) before merging into main.
hotfix/*	Branches cut from main to urgently patch production issues (e.g., hotfix/mqtt-reconnect-bug) without waiting for the next release.

4.2 Justification

- Multiple independent components (backend, AI engine, frontend, IoT broker) are developed in parallel - feature branches isolate this work and prevent conflicts.

- main must always be deployable, since it directly maps to the Docker images used in production deployment.
- release/* branches allow final integration testing of the full Docker Compose stack before a version is shipped.
- hotfix/* branches let critical bugs (e.g., a fault-detection false positive) be patched in production quickly without merging unfinished develop work.

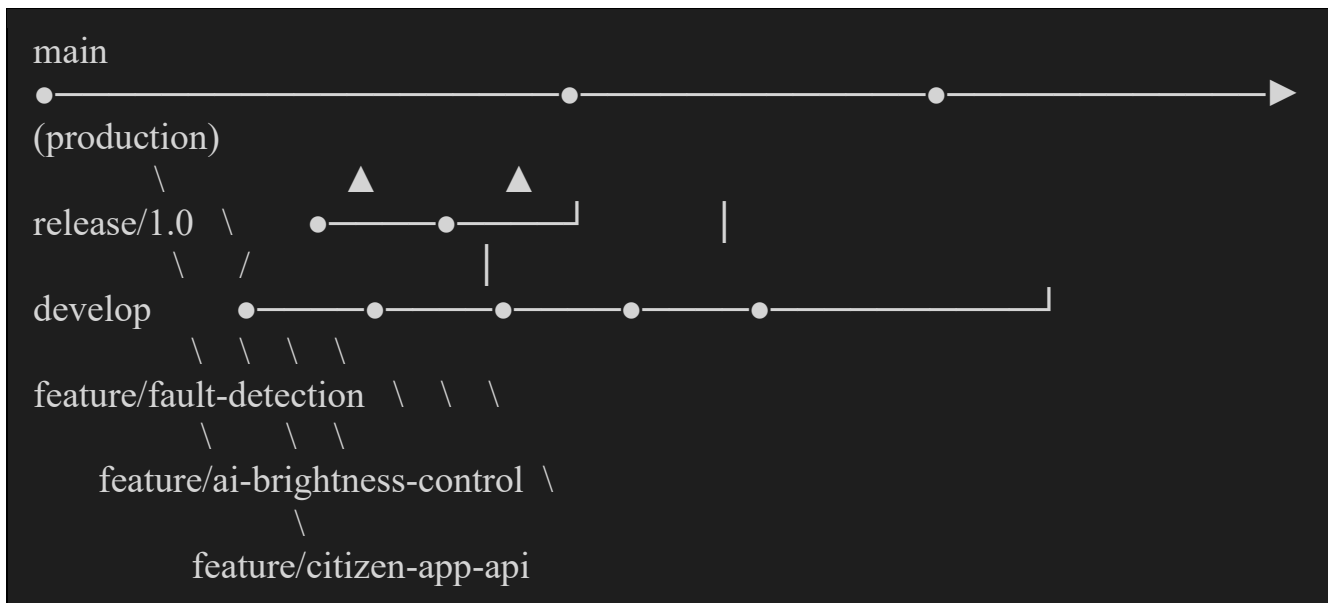


Figure 1: Gitflow branching model applied to the project

5. Version Control Workflow

The following sequence demonstrates a realistic development cycle: creating a feature branch, committing meaningful changes, synchronizing with the remote, merging into develop, and resolving a conflict.

5.1 Feature Development and Commits

```
# Create and switch to a new feature branch off develop
git checkout develop
git pull origin develop
git checkout -b feature/fault-detection

# Work on the fault-detection service, then stage and commit
git add ai-engine/src/fault_detection.py
git commit -m "feat(ai-engine): implement rule-based fault classification"
```

```
git add backend/src/controllers/faultController.js
git commit -m "feat(backend): add endpoint to log detected faults"

git add backend/tests/fault.test.js
git commit -m "test(backend): add unit tests for fault logging endpoint"

# Push the feature branch to the remote for review
git push -u origin feature/fault-detection
```

5.2 Merging via Pull Request

```
# After code review and approval, merge feature into develop
git checkout develop
git pull origin develop
git merge --no-ff feature/fault-detection -m "merge: integrate fault-detection
feature into develop"
git push origin develop

# Delete the feature branch once merged
git branch -d feature/fault-detection
git push origin --delete feature/fault-detection
```

5.3 Conflict Resolution Example

A conflict occurred when two developers modified docker-compose.yml simultaneously - one adding the ai-engine service, another adding the mqtt-broker service.

```
git merge feature/ai-brightness-control
# Auto-merging docker-compose.yml
# CONFLICT (content): Merge conflict in docker-compose.yml
# Automatic merge failed; fix conflicts and then commit the result.

# Open docker-compose.yml, manually combine both service blocks, then:
git add docker-compose.yml
git commit -m "fix: resolve merge conflict in docker-compose.yml (combine ai-
engine and mqtt-broker services)"
git push origin develop
```

5.4 Repository Synchronization

```
# Keep local branch up to date with the remote before starting new work
```

```
git fetch origin
git pull origin develop

# Rebase a long-running feature branch onto the latest develop
git checkout feature/citizen-app-api
git rebase develop
```

5.5 Purpose of Each Git Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for a specific feature.
git add / commit	Stages and records a logical, reviewable unit of change with a descriptive message.
git push -u origin	Publishes the local branch to the remote and sets tracking for future pushes/pulls.
git merge --no-ff	Integrates a completed feature into develop while preserving a merge commit for traceability.
git pull / fetch	Synchronizes the local repository with the latest remote changes to avoid divergence.
git rebase	Replays commits on top of the latest develop to keep history linear and reduce future conflicts.
git branch -d / push --delete	Removes a feature branch locally and remotely once its work is merged, keeping the repo clean.

6. Commit History Analysis

The command below displays a condensed, graphical commit history across branches.

```
git log --oneline --graph --all --decorate
```

```
* 9f3a1c2 (HEAD -> develop) fix: resolve merge conflict in docker-  
compose.yml  
* 7b2e441 merge: integrate fault-detection feature into develop  
\  
| * 4d8f0aa test(backend): add unit tests for fault logging endpoint  
| * 1a6c2e9 feat(backend): add endpoint to log detected faults  
| * e3f9b71 feat(ai-engine): implement rule-based fault classification  
|/  
* c0a4d55 feat(frontend): add real-time status map component  
* 92b1f3d feat(ai-engine): add adaptive brightness control service  
* 5e7a0c8 feat(backend): add streetlight status ingestion API  
* 3f1d9b6 chore: configure Docker Compose for backend and database  
* d0e2a41 chore: initial commit - project scaffolding  
* (main) 0a1b2c3 release: v1.0.0 - initial production release
```

6.1 Good Commit Message Practices Followed

- Conventional Commits prefixes (feat, fix, test, chore, docs, release) clearly indicate the type of change at a glance.
- Each commit message states the scope in parentheses, e.g. feat(ai-engine), so it's clear which component changed.
- Messages describe what changed and, where useful, why (e.g. resolving a specific conflict), aiding future debugging.
- Commits are atomic - each one represents a single logical change rather than bundling unrelated edits together.
- The imperative mood ('add', 'implement', 'resolve') is used consistently, matching Git's own generated messages.

Together, these practices make the history readable, support automated changelog generation, and simplify locating the commit that introduced a regression via git bisect.

7. Docker Environment Design

7.1 Why Docker is Suitable for This Project

- The system has multiple heterogeneous components (Node.js API, Python AI service, React frontend, MQTT broker, PostgreSQL) with different runtimes - Docker isolates each one's dependencies cleanly.
- Containerization guarantees the AI engine's exact Python/ML library versions behave identically in development, testing, and production ('works on my machine' problem eliminated).

- Docker Compose allows the entire stack to be started with a single command, which is essential for demoing and testing IoT-to-dashboard data flow end to end.
- Containers make horizontal scaling straightforward - e.g., running multiple backend API replicas behind a load balancer as the number of connected streetlights grows.
- Portability allows the same images to be deployed to any cloud provider or the municipality's own data center without modification.

7.2 Components Requiring Containerization

Component	Reason for Containerization
backend (Node.js/Express)	Exposes the REST API; needs a consistent Node runtime and npm dependency set.
ai-engine (Python/FastAPI)	Runs ML models for fault detection and brightness optimization; has heavy, version-sensitive dependencies (numpy, scikit-learn, etc.).
frontend (React)	Built as static assets and served via Nginx; benefits from a reproducible, lightweight production image.
PostgreSQL database	Provides persistent storage for streetlight status, faults, and energy logs; run as an official versioned image with a mounted volume.
mqtt-broker (Eclipse Mosquitto)	Receives telemetry from IoT streetlight controllers; runs as an isolated, independently restartable service.
Redis (optional cache)	Caches frequent dashboard queries and session data to reduce database load.

8. Dockerfile Development

8.1 Backend Dockerfile (backend/Dockerfile)

```
# Stage 1: Install
dependencies
FROM node:18-alpine AS
builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
```

```
# Stage 2: Build the
runtime image
FROM node:18-alpine
WORKDIR /app
COPY --from=builder
/app/node_modules
./node_modules
COPY . .
ENV
NODE_ENV=production
EXPOSE 5000
HEALTHCHECK --
interval=30s --timeout=5s
CMD wget -qO-
http://localhost:5000/health
|| exit 1
CMD ["node",
"src/app.js"]
```

Instruction	Purpose
FROM node:18-alpine	Uses a small, official Node.js 18 base image to minimize final image size.
WORKDIR /app	Sets the working directory inside the container for all subsequent instructions.
COPY package*.json ./	Copies only manifest files first so the dependency layer can be cached between builds.
RUN npm ci --omit=dev	Installs exact, locked production dependencies for reproducible builds.
Multi-stage build	Keeps the final image lean by discarding build-only artifacts from the builder stage.
ENV NODE_ENV=production	Configures Express and dependent libraries to run in production mode.
EXPOSE 5000	Documents the port the API listens on.
HEALTHCHECK	Lets Docker/Compose detect and restart an unresponsive backend container automatically.
CMD	Specifies the default command that starts the API server when the container runs.

8.2 AI Engine Dockerfile (ai-engine/Dockerfile)

```
FROM python:3.11-slim
WORKDIR /app

# Install system dependencies required by ML libraries
RUN apt-get update && apt-get install -y --no-install-recommends gcc && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.3 Frontend Dockerfile (frontend/Dockerfile)

```
# Stage 1: Build the React application
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Serve the static build using Nginx
FROM nginx:1.25-alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The frontend uses a multi-stage build: the first stage compiles the React app with Node.js, and the second stage copies only the compiled static files into a lightweight Nginx image, keeping the production image small and free of build tooling.

9. Docker Compose Design

docker-compose.yml orchestrates all services as a single stack, defining networking, volumes, environment variables, and startup dependencies.

version: "3.9"

services:

frontend:

build: ./frontend

ports:

- "3000:80"

depends_on:

- backend

networks:

- streetlight-net

backend:

build: ./backend

ports:

- "5000:5000"

environment:

- NODE_ENV=production

- DB_URL=postgresql://admin:secret@database:5432/streetlight_db

- MQTT_BROKER_URL=mqtt://mqtt-broker:1883

- AI_ENGINE_URL=http://ai-engine:8000

- REDIS_URL=redis://cache:6379

depends_on:

- database

- mqtt-broker

- ai-engine

networks:

- streetlight-net

ai-engine:

build: ./ai-engine

ports:

- "8000:8000"

volumes:

- ai-models:/app/models

networks:

- streetlight-net

database:

image: postgres:15-alpine

restart: always

environment:

- POSTGRES_USER=admin
- POSTGRES_PASSWORD=secret
- POSTGRES_DB=streetlight_db

volumes:

- db-data:/var/lib/postgresql/data
- ./database/init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- streetlight-net

mqtt-broker:

image: eclipse-mosquitto:2

ports:

- "1883:1883"

volumes:

- ./mqtt-broker/mosquitto.conf:/mosquitto/config/mosquitto.conf

networks:

- streetlight-net

cache:

image: redis:7-alpine

networks:

- streetlight-net

networks:

streetlight-net:

driver: bridge

volumes:

db-data:

ai-models:

9.1 Configuration Explained

Element	Explanation
services	Six services are defined: frontend, backend, ai-engine, database, mqtt-broker, and cache - each mapped to its own container.

Element	Explanation
networks (streetlight-net)	A custom bridge network lets services reach each other by service name (e.g., backend connects to database:5432) while isolating them from the host.
volumes (db-data, ai-models)	Named volumes persist PostgreSQL data and trained ML model files across container restarts and rebuilds.
environment variables	Pass configuration (DB connection string, broker URL, AI engine URL) into containers without hardcoding values in source code.
depends_on	Ensures the backend only starts after the database, broker, and AI engine containers have been created, avoiding startup race conditions.
ports	Maps container ports to host ports (e.g., 3000:80 for the dashboard, 5000:5000 for the API) for external access.

10. Container Deployment and Testing

10.1 Building and Running the Stack

```
# Build images for all services and start the stack in detached mode
docker compose build
docker compose up -d
```

```
# Verify all containers are running
docker compose ps
```

```
# NAME                STATUS              PORTS
# streetlight-frontend Up 2 minutes       0.0.0.0:3000->80/tcp
# streetlight-backend  Up 2 minutes       0.0.0.0:5000->5000/tcp
# streetlight-ai-engine Up 2 minutes       0.0.0.0:8000->8000/tcp
# streetlight-database Up 2 minutes       0.0.0.0:5432->5432/tcp
# streetlight-mqtt-broker Up 2 minutes      0.0.0.0:1883->1883/tcp
# streetlight-cache    Up 2 minutes       6379/tcp
```

10.2 Testing Procedure

1. Health check: confirm the backend API responds successfully.

```
curl http://localhost:5000/health
```

```
# {"status":"ok","uptime":"125s"}
```

2. AI engine check: confirm the FastAPI service is reachable.

```
curl http://localhost:8000/docs  
# Returns the FastAPI interactive documentation page (HTTP 200)
```

3. Simulated IoT data: publish a test status message to the MQTT broker.

```
mosquitto_pub -h localhost -t streetlight/1024/status -m '{"lux":0,"state":"ON"}'
```

4. Fault detection: confirm the backend logged a fault from the simulated zero-lux reading.

```
curl http://localhost:5000/api/faults?lightId=1024  
# [{"lightId":1024,"type":"No Output","detectedAt":"2026-08-13T10:15:32Z"}]
```

5. Dashboard check: open <http://localhost:3000> in a browser and confirm the map marker for light 1024 turns red.

6. Container logs: inspect logs for errors during the test run.

```
docker compose logs backend --tail=50  
docker compose logs ai-engine --tail=50
```

10.3 Evidence of Successful Execution

Screenshots captured during testing (to be inserted in the final report) include: (1) the output of `docker compose ps` showing all six containers healthy, (2) the browser dashboard displaying live streetlight markers, (3) the terminal output of the `curl` health checks, and (4) the fault alert notification received after the simulated MQTT publish.

11. Benefits of Git and Docker

Area	Benefit to This Project
Collaboration	Gitflow branches let backend, AI, and frontend teams work in parallel on feature/* branches without blocking each other, merging cleanly through develop.
Version Management	Every change to the fault-detection logic or Dockerfiles is tracked, reviewable, and revertible, which matters when tuning AI thresholds affecting public safety.
Portability	Docker images run identically on a developer's laptop, the CI pipeline, and the municipality's production servers, regardless of underlying OS.

Area	Benefit to This Project
Deployment	docker compose up -d deploys the entire six-service stack in minutes instead of manually installing Node, Python, PostgreSQL, and Mosquitto on each machine.
Scalability	Backend and AI-engine containers can be replicated horizontally (docker compose up --scale backend=3) as more streetlights and citizens are onboarded.
Maintainability	Isolated containers mean an AI-engine dependency upgrade cannot silently break the backend or frontend, simplifying long-term upkeep.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts in docker-compose.yml occurred when multiple developers added services simultaneously, requiring careful manual reconciliation of the networking and volumes sections.
- Container startup ordering: the backend occasionally attempted to connect to PostgreSQL before it had finished initializing, even with depends_on configured (which only waits for container start, not readiness).
- The AI engine's Docker image was initially large (over 1.5 GB) due to unpruned ML library caches, slowing down builds and deployments.
- Managing environment-specific secrets (database passwords, JWT keys) safely across local, staging, and production Compose files required extra discipline.
- Debugging cross-container networking issues (e.g., MQTT broker unreachable from backend) was harder than debugging a single local process.

12.2 Suggested Improvements and Future Enhancements

- Introduce a wait-for-it or healthcheck-based startup script so the backend retries its database connection until PostgreSQL is truly ready, not just started.
- Add a CI/CD pipeline (GitHub Actions) that automatically builds images, runs tests, and lints Dockerfiles on every pull request before merging into develop.
- Adopt a secrets manager (Docker Secrets or HashiCorp Vault) instead of plain environment variables for production credentials.
- Reduce the AI-engine image size further using slimmer base images and multi-stage builds to strip build-only Python packages.

- Introduce Kubernetes for production orchestration once the number of streetlights grows large enough to require automated scaling and self-healing beyond what Docker Compose offers.
- Add automated integration tests that spin up the full Compose stack in CI and verify the IoT-to-dashboard data flow end to end.